

Materia: Programación y Estructura de Datos (COM104)

Profesor: Francisco Javier Aguilar Juárez

Ciclo: 1258

Nombre del proyecto:

Sistema Bancario con Implementación de Estructura de Datos

Miembros del Equipo		
ID	Nombre (ap.Paterno, ap.Materno, nombre)	Carrera
0270653	Betancourt Quintana Santiago	LIAVC
0230817	Gutiérrez Lara Elva María	LIDCI
0275460	Cuéllar Rivera Jesús Roberto	LIAVC
0270859	Sánchez Antonio Máximo Samar	LIAVC

ÍNDICE / INDEX

- I.** Introducción / *Introduction* → 2

- II.** Descripción del Problema / *Problem Description* → 2

- III.** Diseño: Tipo de Dato Abstracto (TDA) / *Design: Abstract Data Type (ADT)* → 3

- IV.** Explicación de Cómo el Diseño Resuelve el Problema / *Explanation of How Design Solves the Problem* → 7

- V.** Implementación del Código / *Code Implementation* → 10

- VI.** Prueba de Imagen / *Image Proof* → 15

- VII.** Guía de Usuario / *User Guide* → 26

I. INTRODUCCIÓN / INTRODUCTION

Se presenta un sistema de banco, el cual puede utilizarse como cliente o como empleado. El cliente puede realizar distintas operaciones, como transacciones, depositar, pedir un préstamo, entre otros. Mientras que el empleado se encarga de revisar que las operaciones de los clientes se realicen de manera correcta. El empleado igualmente puede dar de alta a nuevos clientes, y abrir una fila adicional, en caso de que todas las filas estén saturadas. El proyecto propuesto, implementa distintas estructuras de datos aprendidas durante el curso de Programación y Estructura de Datos. Las estructuras implementadas son: lista doblemente enlazada, cola, pila y árboles binarios.

A bank system is presented, which can be used both as a client or as an employee. The client will be able to perform different operations such as transactions, deposits, requesting loans, among others. On the other hand, the employee will make sure every operation made by the client is done correctly. Additionally, the employee will be able to register new clients into the system, as well as open a new queue in case the other ones are full. The data structures implemented are: double linked lists, queues, stack, and binary trees.

II. DESCRIPCIÓN DEL PROBLEMA / PROBLEM DESCRIPTION

Implementar un sistema bancario que permita organizar, automatizar y optimizar la atención a clientes. Surge como necesidad para las instituciones financieras ubicadas en las áreas rurales que siguen teniendo deficiencias en la gestión de la información y en los procesos de atención al público. En muchas de estas sucursales las operaciones bancarias, es decir, retiros, depósitos, préstamos, entre otros se realizan de forma manual o con herramientas limitadas. Su forma de realizarse provoca desorganización, lentitud en el servicio y errores en el registro de la información.

Actualmente, la falta de un sistema digital que gestione automáticamente las filas de atención, asignación de turnos, etc, genera problemas tanto para los empleados como los usuarios. Los clientes suelen desconocer el tiempo estimado para ser atendidos, lo que ocasiona inconformidad y pérdida de tiempo. Por otro lado, los empleados enfrentan dificultades para atender múltiples solicitudes de manera ordenada, ya que carece de un método automatizado.

Frente a la problemática planteada, se propone implementar un sistema bancario con estructuras dinámicas, capaz de simular los procesos internos de una sucursal. El sistema propuesto debe de organizar filas por tipo de servicio (retiros, depósitos, préstamos, etc.), asignar turnos automáticamente, registrar operaciones de manera segura y estructurada, en un entorno digital sencillo de usar para el usuario y el personal. con la implementación se busca reducir los tiempos de fila para los clientes, minimizar errores y mejorar la gestión de información, ofreciendo una herramienta accesible para instituciones rurales, que no cuentan con sistemas de atención automatizados.

Implementing a banking system that allows the organization, automation, and optimization of customer service arises as a necessity for financial institutions located in rural areas, which continue to face deficiencies in information management and customer service processes. In many of these branches, banking operations —such as withdrawals, deposits, and loans— are carried out manually or through limited tools. This method of operation leads to disorganization, slow service, and errors in information recording.

Currently, the absence of a digital system that automatically manages service queues and turn assignment generates problems for both employees and clients. Customers often do not know the estimated time they will be served, which causes dissatisfaction and loss of time. On the other hand, employees face difficulties in managing multiple service requests in an orderly manner since they lack an automated method to handle them.

In response to this situation, it is proposed to implement a banking system with dynamic data structures, capable of simulating the internal processes of a branch. The proposed system should organize service queues by type of operation (withdrawals, deposits, loans, etc.), assign turns automatically, and record operations securely and in an organized manner, all within a digital environment that is easy to use for both clients and staff. With this implementation, the goal is to reduce customer waiting times, minimize human errors, and improve information management, providing an accessible and efficient tool for rural institutions that do not yet have automated customer service systems.

III. DISEÑO- TDA/

En el desarrollo de este proyecto de sistema bancario en C++, los TDAs son cruciales para modelar entidades complejas y procesos dinámicos, como el historial de transacciones o la

gestión de la cola de espera de clientes. Además de que podemos separar la interfaz de la implementación por medio de la encapsulación de estructuras dinámicas (nodos y punteros) dentro de las clases.

El sistema se basa en cuatro TDAs principales:

1. Pila con estructura LIFO (Last In, First Out)

COMPONENTE	IMPLEMENTACIÓN	ROL
Clase Contenedora	Cliente	Almacena y gestiona el historial de transacciones de una cuenta.
Estructura Interna	struct NodoPila, puntero tope	Define la estructura del nodo en la memoria dinámica.
Operación Push (Apilar)	Cliente::AnhadirTransaccion(Transaccion)	Registra una nueva transacción en el historial.
Operación Pop (Desapilar)	Cliente::EliminarTransaccion()	Permite revertir la última operación, fundamental para la gestión de Problemas.

2. Fila FIFO (First In, First Out)

COMPONENTE	IMPLEMENTACIÓN	ROL
Clase Contenedora	FilaClientes	Simula la cola de espera de clientes en una sucursal.
Estructura Interna	struct NodoFila, punteros inicio y fin	Controla quién entra (fin) y quién sale (inicio) de la cola.
Operación Enqueue (Encolar)	FilaClientes::AgregarCliente(Cliente)	Coloca a un cliente al final de la fila.
Operación Dequeue (Desencolar)	Dequeue (Desencolar)FilaClientes::EliminarClienteFila()	Extrae al cliente más antiguo de la fila para ser atendido (FilaClientes::AtenderCliente).

3. Árbol Binario de Búsqueda

COMPONENTE	IMPLEMENTACIÓN	ROL
Clase Contenedora	Empleado	Actúa como la base de datos central de clientes.
Estructura Interna	struct NodoArbol, puntero raiz	Almacena todos los objetos Cliente, ordenados por su ID único.
Operación Inserción	Empleado::InsertarCliente (Cliente)	Creación de una nueva cuenta de cliente.
Operación Búsqueda	Empleado::BuscarCliente (int idBuscar)	Localización rápida de un cliente para login o depósitos.
Operación Actualización	Empleado::ActualizarCliente (Cliente)	Persistencia de cambios (saldo, historial) después de una transacción.

4. Lista Doblemente Enlazada

COMPONENTE	IMPLEMENTACIÓN	ROL
Clase Contenedora	Cliente	Gestiona la lista de contactos del cliente.
Estructura Interna	struct NodoListaDoble, punteros cabeza y cola	Almacena punteros a otros objetos Cliente (los contactos).
Operaciones	Cliente::insertarContacto(), Cliente::eliminarContacto()	Permite gestionar de manera eficiente la lista de contactos para realizar depósitos.

In the development of this project on C ++ , the ADT of each structure was crucial to model the complex structures and dynamic procedures, like the transaction history or managing

clients, and controlling service queues. Moreover the interface and the implementation can be separated by encapsulating the dynamic structures (nodes and pointers) inside the classes.

The system is designed around four main Abstract Data Types (ADTs).

1. Stack with LIFO (Last In, First Out)

COMPONENT	IMPLEMENTATION	ROLE
Container Class	Cliente	Stores and manages the transaction history of an account.
Internal Structure	struct NodoPila, puntero tope	Defines the node structure in dynamic memory.
Push Operation (Apilar)	Cliente::AnhadirTransaccion(Transaccion)	Registers a new transaction in history.
Pop Operation (Desapilar)	Cliente::EliminarTransaccion()	Allows reverting the last operation, which is essential for handling the Problems service.

2. Queue FIFO (First In, First Out)

COMPONENT	IMPLEMENTATION	ROLE
Container Class	FilaClientes	Simulates the waiting line of clients in a bank branch.
Internal Structure	struct NodoFila, punteros inicio y fin	Controls who enters (rear) and who leaves (front) of the queue.
Enqueue Operation (Encolar)	FilaClientes::AgregarCliente(Cliente)	Adds a client to the queue.
Dequeue Operation (Desencolar)	Dequeue (Desencolar) FilaClientes::EliminarClienteFila()	Removes the oldest client from the queue, that is now on its turn being assisted

3. Binary Search Tree (BST)

COMPONENT	IMPLEMENTATION	ROLE
Container Class	Empleado	Acts as the central database
Internal Structure	struct NodoArbol, puntero raiz	Stores all clients (objects), ordered by their unique ID
Insertion Operation	Empleado::InsertarCliente (Cliente)	Creates a new account for the client
Search Operation	Empleado::BuscarCliente (int idBuscar)	Quickly locate a client for login or deposit
Update Operation	Empleado::ActualizarClien te(Cliente)	Ensures data persistence after a transaction (balance history updates)

4. Double Linked List

COMPONENT	IMPLEMENTATION	ROLE
Container Class	Cliente	Manages the clients contacts list
Internal Structure	struct NodoListaDoble, punteros cabeza y cola	Stores pointers to other clients (objects) which act as contacts
Operations	Cliente::insertarContacto(), Cliente::eliminarContacto()	Allows efficient management of the contact list to perform a deposit operations

IV. EXPLICACIÓN DE COMO EL DISEÑO RESUELVE EL PROBLEMA / **EXPLANATION OF HOW THE DESIGN SOLVES THE PROBLEM**

El sistema bancario “Banco Javi” lo desarrollamos basándonos en el TDA definido anteriormente. Con esto pudimos organizar los procesos internos de la sucursal de una forma clara y manejable. Desde el inicio decidimos trabajar con clases y objetos en lugar de *structs*,

porque necesitábamos que los datos no solo se almacenaran, sino que también tuvieran comportamiento propio. A diferencia de los *structs* o los arreglos estáticos, la orientación a objetos nos permitió que cada parte del sistema funcionara con su propia lógica, evitando saturar el “centro” del programa y dejando todo más ordenado.

Para atacar el problema del desorden y la lentitud en la atención —sobre todo en sucursales rurales donde el flujo suele volverse caótico— implementamos la clase *FilaClientes*, que sigue el principio FIFO de manera estricta. Con esto logramos que el orden de llegada se respete sin depender de que alguien esté organizando manualmente. Además, dividimos las filas por tipo de trámite: retiros, depósitos, préstamos y problemas. Esto evita confusiones y hace que cada área avance más rápido porque cada empleado se dedica a un solo tipo de solicitud.

También queríamos resolver los tiempos muertos causados por la búsqueda manual de información. Para eso integramos un Árbol Binario de Búsqueda (ABB) dentro de la clase *Empleado*. Esto nos permite localizar a un cliente por su ID de forma mucho más rápida que con una búsqueda tradicional. Así, el empleado puede acceder al perfil y al saldo del cliente casi de inmediato, lo cual agiliza todo el servicio.

Por otro lado, para reducir errores y darle más seguridad al sistema, cada objeto *Cliente* incluye una Pila (Stack) que funciona como un registro interno de movimientos. Gracias al principio LIFO podemos revertir el último cambio si algo se registró mal, restaurando el saldo sin complicaciones. Esta parte fue clave para evitar los típicos errores humanos que surgen en operaciones rápidas.

Finalmente, para facilitar las operaciones recurrentes entre usuarios, añadimos una Lista Doblemente Enlazada para manejar contactos frecuentes de manera sencilla.

En conjunto, el sistema no solo digitaliza los procesos, sino que realmente organiza y agiliza el funcionamiento de la sucursal. Al dejar atrás métodos estáticos y manuales y reemplazarlos con estructuras dinámicas, logramos una plataforma más eficiente, confiable y adaptable a las necesidades de las sucursales rurales.

Our team developed the “Banco Javi” banking system based on the TDA we had previously defined. This helped us organize the internal processes of the branch in a clear and

manageable way. From the beginning, we chose to work with classes and objects instead of structs, because we needed the data not only to be stored but also to have their own behavior. Unlike structs or static arrays, object-oriented design allowed each part of the system to operate with its own logic, preventing the core of the program from becoming overloaded and keeping everything better structured.

To address the issue of disorder and slow customer service —especially in rural branches where queues often become chaotic— we implemented the *FilaClientes* class, which follows the FIFO principle strictly. This ensures that the order of arrival is respected without relying on someone managing the line manually. We also separated the queues by type of service: withdrawals, deposits, loans, and issues. This prevents confusion and makes each area run faster since each employee focuses on only one type of transaction.

We also wanted to eliminate downtime caused by manual information lookup. To solve this, we integrated a Binary Search Tree (BST) inside the *Empleado* class. This allows us to find a customer by their ID much faster than with a sequential search. As a result, the employee can access the customer's profile and balance almost instantly, which speeds up the entire service.

To reduce errors and increase the system's reliability, each *Cliente* object includes a *Pila* (Stack) that works as an internal log of operations. Because it follows the LIFO principle, we can revert the last operation if something was recorded incorrectly, restoring the balance without complications. This was essential to minimize the typical human errors that occur during fast transactions.

Finally, to make recurring actions easier for users, we added a Doubly Linked List to manage frequent contacts more conveniently.

Overall, the system doesn't just digitalize processes —it actually organizes and speeds up the day-to-day operation of the branch. By moving away from static and manual methods and replacing them with dynamic structures, we built a platform that is more efficient, reliable, and adaptable to the needs of rural financial institutions.

V. IMPLEMENTACIÓN DEL CÓDIGO

I. Estructuras de Datos Centrales (TDAs)

El sistema se fundamenta en la encapsulación de cuatro TDAs para modelar procesos y almacenamiento eficientemente:

- **Árbol Binario de Búsqueda (ABB):**
 - **Clase Contenedora:** Empleado.
 - **Propósito:** Servir como la Base de Datos Central de todos los clientes, permitiendo la búsqueda, inserción y actualización de cuentas de forma rápida (ordenado por ID).
- **Fila (Queue):**
 - **Clase Contenedora:** FilaClientes.
 - **Propósito:** Modelar la Cola de Atención del banco. Opera bajo el principio **FIFO** (*First In, First Out*) para procesar transacciones en orden de llegada.
- **Pila (Stack):**
 - **Clase Contenedora:** Cliente.
 - **Propósito:** Gestionar el historial de transacciones de la cuenta. Opera bajo el principio **LIFO** (*Last In, First Out*), lo que facilita la reversión de la última operación en caso de Problemas.
- **Lista Doblemente Enlazada:**
 - **Clase Contenedora:** Cliente.
 - **Propósito:** Almacenar y gestionar la lista de contactos del cliente para realizar depósitos a cuentas conocidas.

II. Clase Cliente (Entidad de Cuenta)

La clase Cliente (Cliente.h y Cliente.cpp) encapsula la información de una cuenta y su actividad financiera.

1. Atributos Fundamentales:

- **ID:** Generado estáticamente e incrementalmente (inicia en 10000) para garantizar unicidad.
- **dinero:** Saldo de la cuenta (float).
- **Contraseña y nombre:** Datos de acceso y personales.

- **prestamo:** Una estructura Transaccion que guarda el monto pendiente de un préstamo.

2. Métodos de Pila (Historial):

- **AnhadirTransaccion(Transaccion):** Implementa la operación push para registrar un movimiento en el historial.
- **EliminarTransaccion():** Implementa la operación pop. Desapila la última operación y contiene la lógica para **revertir su efecto en el saldo** (dinero).
- **EstadoCuenta():** Recorre la pila para mostrar el historial completo.

3. Métodos de Lista Doble (Contactos):

- **insertarContacto(Cliente):** Añade un puntero a otro objeto Cliente a la lista.
- **eliminarContacto(string):** Busca y elimina un contacto de la lista.

III. Clase Empleado (Administración y Base de Datos)

La clase Empleado (Empleado.h y Empleado.cpp) actúa como el gestor central del banco.

1. Funciones del Árbol Binario de Búsqueda (ABB):

- **InsertarCliente(Cliente&):** Añade un nuevo cliente a la base de datos, manteniendo la estructura ABB ordenada por ID.
- **BuscarCliente(int idBuscar):** Realiza búsquedas rápidas para login y depósitos.
- **ActualizarCliente(Cliente&):** Localiza el nodo por ID y sobrescribe los datos del cliente (saldo, historial, contactos), asegurando la **persistencia** de los cambios en el ABB.

2. Gestión de Colas de Atención:

- El Empleado contiene cuatro instancias privadas de la clase FilaClientes (una para Retiros, Depósitos, Préstamos y Problemas).
- **FormarFila(Cliente&, short nofila):** Asigna al cliente a la cola de atención (FilaClientes) correspondiente según el servicio solicitado.

IV. Clase FilaClientes (Procesamiento de Transacciones)

La clase FilaClientes (FilaClientes.h y FilaClientes.cpp) es responsable de la ejecución efectiva de las operaciones financieras.

1. Implementación de la Fila (Cola):

- **AgregarCliente(Cliente):** Enlaza un nuevo nodo al puntero fin.

- **EliminarClienteFila():** Retira el nodo del puntero inicio.
- 2. **Motor de Lógica de Negocio:**
 - **AtenderCliente(Transaccion, Empleado & empleado):**
 - Es el punto donde se ejecuta el movimiento de dinero.
 - Extrae al primer cliente de la fila.
 - Utiliza un switch para aplicar la lógica específica: validar saldo, restar monto, sumar préstamo, o buscar un destinatario para depósitos.
 - Al finalizar la lógica, llama obligatoriamente a empleado.ActualizarCliente() para guardar los cambios de saldo e historial del cliente de vuelta en el ABB.

V. Flujo General del Programa (main.cpp)

El flujo simula la interacción del usuario y el sistema:

1. **Inicio:** El programa crea la instancia central: Empleado empleado1;
2. **Rol Cliente:**
 - El usuario se autentica (empleado.BuscarCliente).
 - Al solicitar una transacción, el cliente NO mueve el dinero, solo llama a empleado.FormarFila(...).
3. **Rol Empleado:**
 - El empleado selecciona una fila de atención.
 - Llama a empleado.AtenderCliente().
 - Esto activa FilaClientes::AtenderCliente, que ejecuta la lógica transaccional y persiste los cambios en el ABB, completando el ciclo de la transacción.

I. Core Data Structures ("TDAs")

The system relies on the encapsulation of four ADTs to efficiently model processes and storage.

- **"Árbol Binario de Búsqueda" (ABB - Binary Search Tree):**
 - **Containing Class: "Empleado".**
 - **Purpose:** Serves as the **Central Database** for all clients, enabling fast search, insertion, and updating of accounts (sorted by **ID**).

- **"Fila" (Queue):**
 - **Containing Class:** "FilaClientes".
 - **Purpose:** Models the bank's **Customer Service Queue**. Operates under the **FIFO** (*First In, First Out*) principle to process transactions in order of arrival.
- **"Pila" (Stack):**
 - **Containing Class:** "Cliente".
 - **Purpose:** Manages the account's **transaction history**. Operates under the **LIFO** (*Last In, First Out*) principle, which facilitates reversing the last operation in case of "Problemas" (Issues).
- **"Lista Doblemente Enlazada" (Doubly Linked List):**
 - **Containing Class:** "Cliente".
 - **Purpose:** Stores and manages the client's **contact list** for known accounts, used for deposits.

II. "Clase Cliente" (Account Entity)

The **"Clase Cliente"** ("Cliente.h" and "Cliente.cpp") encapsulates account information and financial activity.

1. **Fundamental Attributes:**
 - **"ID"**: Statically and incrementally generated (starting at 10000) to ensure uniqueness.
 - **"dinero"**: The account balance (**float**).
 - **"Contraseña"** and **"nombre"**: Access credentials and personal data.
 - **"prestamo"**: A **"Transaccion"** structure that tracks the pending amount of a loan.
2. **"Pila" (Stack) Methods (History):**
 - **"AnhadirTransaccion"** ("Transaccion"): Implements the **push** operation to record a movement in the history.
 - **"EliminarTransaccion"**: Implements the **pop** operation. It unshelves the last operation and contains the logic to **revert its effect on the "dinero"** (balance).
 - **"EstadoCuenta"**: Traverses the stack to display the complete history.
3. **"Lista Doble" (Doubly Linked List) Methods (Contacts):**

- **"insertarContacto"** ("Cliente"): Adds a pointer to another **"Cliente"** object to the list.
- **"eliminarContacto"** (**string**): Searches for and removes a contact from the list.

III. "Clase Empleado" (Administration and Database)

The **"Clase Empleado"** ("Empleado.h" and "Empleado.cpp") acts as the bank's central manager.

1. "Árbol Binario de Búsqueda" (ABB) Functions:

- **"InsertarCliente"** (&"Cliente"): Adds a new client to the database, maintaining the ABB structure ordered by **ID**.
- **"BuscarCliente"** (**int idBuscar**): Performs fast searches for login and deposits.
- **"ActualizarCliente"** (&"Cliente"): Locates the node by **ID** and overwrites the client's data (balance, history, contacts), ensuring the **persistence of changes** in the ABB.

2. Service Queue Management:

- The **"Empleado"** contains four private instances of the **"Clase FilaClientes"** (e.g., **"filaClientesRetiros"**) to segment service requests.
- **"FormarFila"** (&"Cliente", **short nofila**): Assigns the client to the corresponding service queue (**"FilaClientes"**) based on the requested service.

IV. "Clase FilaClientes" (Transaction Processing)

The **"Clase FilaClientes"** ("FilaClientes.h" and "FilaClientes.cpp") is responsible for the effective execution of financial operations.

1. "Fila" (Queue) Implementation:

- **"AgregarCliente"** ("Cliente"): Links a new node to the **"fin"** (end) pointer.
- **"EliminarClienteFila"**: Removes the node from the **"inicio"** (beginning) pointer.

2. Business Logic Engine:

- **"AtenderCliente"** ("Transaccion", &"Empleado"):
 - This is the point where the **movement of money is executed**.

- It extracts the first client from the queue.
- It uses a **switch** statement to apply the specific logic: validate balance, subtract amount, add loan, or search for a recipient for deposits.
- After the logic is complete, it **must** call **"empleado.ActualizarCliente()"** to save the updated balance and history back into the ABB.

V. General Program Flow ("main.cpp")

The flow simulates the user interaction and the system's processing cycle:

1. **Start-up:** The program creates the central instance: **"Empleado empleado1"**.
2. **Client Role:**
 - The user authenticates (**"empleado.BuscarCliente"**).
 - When requesting a transaction, the client **DOES NOT** move the money; they only call **"empleado.FormarFila(...)"**.
3. **Employee Role:**
 - The employee selects a service queue.
 - They call **"empleado.AtenderCliente()"**.
 - This activates **"FilaClientes::AtenderCliente"**, which executes the transactional logic and **persists the changes in the ABB**, completing the transaction cycle.

6. PRUEBA DE IMAGEN / IMAGING PROOF

1. Menú Principal del Sistema bancario

```

-----BIENVENIDO AL SISTEMA BANCARIO-----

Seleccione su rol:
    1. Cliente
    2. Empleado
    3. Salir
Opción:

```


2. Menú del cliente

```
-----MENU CLIENTE-----  
  
1)      Iniciar Sesión  
  
2)      Crear Cuenta  
  
3)      Volver al Menú Principal
```

3. Creación de cuenta de cliente

```
2  
      Nos da gusto que desee crear una cuenta.  
Ingrese los siguientes datos:  
      Nombre: Ana  
  
Dinero inicial a depositar: 1220  
  
Ingrese la contraseña deseada: 1  
  
Creando cuenta para Ana con un depósito inicial de $1220...  
  
Cuenta creada exitosamente para Ana!  
Su id es 10001
```

4. Submenú tras haber iniciado sesión

```
Bienvenido al submenú de cliente, Ana!  
1) Ver estado de cuenta  
2) Realizar transacción  
3) Agregar Contacto  
4) Eliminar contacto  
5) Mostrar Contactos  
6) Salir  
Seleccione una opción: |
```

5. Selección de Estado de cuenta

```
Seleccione una opción: 1  
      ESTADO DE CUENTA  
      Pila de Transacciones  


| Tipo     | Monto |
|----------|-------|
| Prestamo | 400   |

  
      Resumen de transacciones:  
      Retiros: 0  
      Depositos: 0  
      Abonos en Efectivo: 0  
      Prestamo pendiente: 400  
Monto disponible: 1620
```

6. Selección- Realizar Transacción

```
          Qué tipo de transacción desea realizar?
1) Retiro
2) Depósito
3) Abono en efectivo
4) Préstamo
5) Eliminar transaccion
Seleccione una opción: |
```

7. Realizar Transacción - Retiro

```
=====
                        BANCO JAVI S.A. DE C.V.
-----
                        TICKET DE TURNO
-----
CLIENTE : Ana
ID       : 10001
TURNO    : 1
HORA ESTIMADA: 23:05 hrs
-----
                Gracias por su preferencia. Vuelva pronto.
=====

Se ha formado en la fila de Retiros.
```

8. Realizar Transacción - Depósito

```
=====
                        BANCO JAVI S.A. DE C.V.
-----
                        TICKET DE TURNO
-----
CLIENTE : Ana
ID       : 10001
TURNO    : 2
HORA ESTIMADA: 23:11 hrs
-----
                Gracias por su preferencia. Vuelva pronto.
=====

Se ha formado en la fila de Depósitos.
```

9. Realizar Transacción - Préstamo

```
=====
                        BANCO JAVI S.A. DE C.V.
=====
                        TICKET DE TURNO
=====
CLIENTE : Ana
ID      : 10001
TURNO   : 3
HORA ESTIMADA: 23:17 hrs
=====
                        Gracias por su preferencia. Vuelva pronto.
=====

Se ha formado en la fila de Préstamos.
```

10. Realizar Transacción - Problemas

```
=====
                        BANCO JAVI S.A. DE C.V.
=====
                        TICKET DE TURNO
=====
CLIENTE : Ana
ID      : 10001
TURNO   : 6
HORA ESTIMADA: 23:34 hrs
=====
                        Gracias por su preferencia. Vuelva pronto.
=====

Se ha formado en la fila de Problemas.
```

11. Agregar Contacto

```
Seleccione una opción: 3
      Ingrese el id del nuevo contacto: 10016
      Contacto agregado exitosamente.
```

12. Mostrar Contacto(s)

```
Seleccione una opción: 5
Contacto 1:

Nombre: Diego
ID: 10016
-----
Contacto 2:

Nombre: Juan
ID: 10010
-----
Contacto 3:

Nombre: Jorge
ID: 10007
-----
```

13. Menú del empleado

```
Ingrese su id:
275460

Ingrese su contraseña:
85696

                                BIENVENIDO AL SISTEMA BANCARIO
1) Atender clientes
2) Mostrar filas
3) Salir
Seleccione una opción: |
```

14. Atender Clientes - Depósito

```
1) Atender Retiro
2) Atender Depósito
3) Atender Abono Efectivo
4) Atender Prestamo
5) Atender Problemas
Elija una opción: 2

=====
                        BANCO JAVI S.A. DE C.V.
=====
                        COMPROBANTE DE ATENCION
=====
TURNO ATENDIDO : 1
CLIENTE       : Ana
ID            : 10001
HORA ATENCION : 23:14 hrs
=====

                        Atendiendo depósito para el cliente: Ana
                        Ingrese el ID del destinatario: 10007
                        Ingrese la cantidad a depositar: 123
                        Depósito exitoso. Nuevo saldo: 1097
=====

TURNO 1 ATENDIDO CON ÉXITO.
FECHA: 19/11/2025
=====

                        Gracias por su preferencia. Vuelva pronto.
=====
```

15. Atender Clientes - Préstamo

```
Eliga una opción: 4

=====
BANCO JAVI S.A. DE C.V.
=====
COMPROBANTE DE ATENCION
=====
TURNO ATENDIDO : 2
CLIENTE       : Ana
ID            : 10001
HORA ATENCION : 23:18 hrs
=====
Atendiendo préstamo para el cliente: Ana
No hay préstamo pendiente. Préstamo aceptado.
Ingrese el monto del préstamo: 400
Ingrese el plazo de pago en días: 50
Fecha de vencimiento: 8/1/2026
Préstamo otorgado. Nuevo saldo: 1620
=====
TURNO 2 ATENDIDO CON ÉXITO.
FECHA: 19/11/2025
=====
Gracias por su preferencia. Vuelva pronto.
=====
```

16. Mostrar Filas

```
Eliga una opción: 2

Cliente 1           Nombre: Ana           ID: 10001

Cliente 2           Nombre: Diego          ID: 10016

Cliente 3           Nombre: Jorge          ID: 10007
```

17. Cierre del sistema

```
3
Volviendo al Menú Principal...

-----BIENVENIDO AL SISTEMA BANCARIO-----

Seleccione su rol:
1. Cliente
2. Empleado
3. Salir
Opción: 3

Gracias por usar el sistema. ¡Hasta luego!
```

1. Main Menu of the Banking System

```
-----BIENVENIDO AL SISTEMA BANCARIO-----  
  
Seleccione su rol:  
    1. Cliente  
    2. Empleado  
    3. Salir  
Opción:
```

2. Client Menu

```
-----MENU CLIENTE-----  
  
1)    Iniciar Sesión  
2)    Crear Cuenta  
3)    Volver al Menú Principal
```

3. Client Account Creation

```
2  
    Nos da gusto que desee crear una cuenta.  
Ingrese los siguientes datos:  
    Nombre: Ana  
  
Dinero inicial a depositar: 1220  
  
Ingrese la contraseña deseada: 1  
  
Creando cuenta para Ana con un depósito inicial de $1220...  
  
Cuenta creada exitosamente para Ana!  
Su id es 10001
```

4. Submenu After Login

```
Bienvenido al submenú de cliente, Ana!  
1) Ver estado de cuenta  
2) Realizar transacción  
3) Agregar Contacto  
4) Eliminar contacto  
5) Mostrar Contactos  
6) Salir  
Seleccione una opción: |
```

5. Account Statement Selection

```
Seleccione una opción: 1
                        ESTADO DE CUENTA
                        Pila de Transacciones
Tipo                   Monto
Prestamo              400

Resumen de transacciones:
Retiros: 0
Depositos: 0
Abonos en Efectivo: 0
Prestamo pendiente: 400
Monto disponible: 1620
```

6. Selection – Perform Transaction

```
Qué tipo de transacción desea realizar?
1) Retiro
2) Depósito
3) Abono en efectivo
4) Préstamo
5) Eliminar transaccion
Seleccione una opción: |
```

7. Perform Transaction – Withdrawal

```
=====
                        BANCO JAVI S.A. DE C.V.
-----
                        TICKET DE TURNO
-----
CLIENTE : Ana
ID       : 10001
TURNO    : 1
HORA ESTIMADA: 23:05 hrs
-----
Gracias por su preferencia. Vuelva pronto.
=====

Se ha formado en la fila de Retiros.
```

8. Perform Transaction – Deposit

```
=====
                        BANCO JAVI S.A. DE C.V.
=====
                        TICKET DE TURNO
=====
CLIENTE : Ana
ID      : 10001
TURNO   : 2
HORA ESTIMADA: 23:11 hrs
=====
                        Gracias por su preferencia. Vuelva pronto.
=====

Se ha formado en la fila de Depósitos.
```

9. Perform Transaction – Loan

```
=====
                        BANCO JAVI S.A. DE C.V.
=====
                        TICKET DE TURNO
=====
CLIENTE : Ana
ID      : 10001
TURNO   : 3
HORA ESTIMADA: 23:17 hrs
=====
                        Gracias por su preferencia. Vuelva pronto.
=====

Se ha formado en la fila de Préstamos.
```

10. Perform Transaction – Issues

```
=====
                        BANCO JAVI S.A. DE C.V.
=====
                        TICKET DE TURNO
=====
CLIENTE : Ana
ID      : 10001
TURNO   : 6
HORA ESTIMADA: 23:34 hrs
=====
                        Gracias por su preferencia. Vuelva pronto.
=====

Se ha formado en la fila de Problemas.
```


11. Add Contact

```
Seleccione una opción: 3
    Ingrese el id del nuevo contacto: 10016
    Contacto agregado exitosamente.
```

12. Show Contact(s)

```
Seleccione una opción: 5
Contacto 1:

Nombre: Diego
ID: 10016
-----
Contacto 2:

Nombre: Juan
ID: 10010
-----
Contacto 3:

Nombre: Jorge
ID: 10007
-----
```

13. Employee Menu

```
Ingrese su id:
275460

Ingrese su contraseña:
85696

                                BIENVENIDO AL SISTEMA BANCARIO
1) Atender clientes
2) Mostrar filas
3) Salir
Seleccione una opción: |
```

14. Attend Clients – Deposit

```
1) Atender Retiro
2) Atender Depósito
3) Atender Abono Efectivo
4) Atender Prestamo
5) Atender Problemas
Elija una opción: 2

=====
                        BANCO JAVI S.A. DE C.V.
=====
                        COMPROBANTE DE ATENCION
=====
TURNO ATENDIDO : 1
CLIENTE       : Ana
ID            : 10001
HORA ATENCION : 23:14 hrs
=====
Atendiendo depósito para el cliente: Ana
Ingrese el ID del destinatario: 10007
Ingrese la cantidad a depositar: 123
Depósito exitoso. Nuevo saldo: 1097
=====
TURNO 1 ATENDIDO CON ÉXITO.
FECHA: 19/11/2025
=====
Gracias por su preferencia. Vuelva pronto.
=====
```

15. Attend Clients – Loan

```
Eliga una opción: 4

=====
BANCO JAVI S.A. DE C.V.
=====
COMPROBANTE DE ATENCION
=====
TURNO ATENDIDO : 2
CLIENTE       : Ana
ID            : 10001
HORA ATENCION  : 23:18 hrs
=====
Atendiendo préstamo para el cliente: Ana
No hay préstamo pendiente. Préstamo aceptado.
Ingrese el monto del préstamo: 400
Ingrese el plazo de pago en días: 50
Fecha de vencimiento: 8/1/2026
Préstamo otorgado. Nuevo saldo: 1620
=====
TURNO 2 ATENDIDO CON ÉXITO.
FECHA: 19/11/2025
=====
Gracias por su preferencia. Vuelva pronto.
=====
```

16. Show Queues

```
Eliga una opción: 2

Cliente 1           Nombre: Ana           ID: 10001

Cliente 2           Nombre: Diego          ID: 10016

Cliente 3           Nombre: Jorge          ID: 10007
```

17. System Shutdown

```
3
Volviendo al Menú Principal...

-----BIENVENIDO AL SISTEMA BANCARIO-----

Seleccione su rol:
1. Cliente
2. Empleado
3. Salir
Opción: 3

Gracias por usar el sistema. ¡Hasta luego!
```

VII. GUÍA DE USUARIO / USER GUIDE

1. Inicio y Selección de Roles

Al iniciar la aplicación, se le pedirá que seleccione su rol:

- **Opción 1: Cliente**
 - Permite crear o acceder a una cuenta para solicitar transacciones.
- **Opción 2: Empleado**
 - Requiere las credenciales predefinidas (**ID_EMPLEADO: 275460**, **CONTRASEÑA: 85696**) para acceder al panel de procesamiento.

2. Guía para el Cliente

Como cliente, su función es gestionar su cuenta y solicitar servicios.

A. Gestión de Cuenta

1. **Crear Cuenta:** Se le asignará un **ID** único y deberá establecer una contraseña. Su cuenta se almacena en la base de datos central (el Árbol Binario de Búsqueda o **ABB**) mediante la función **Empleado::InsertarCliente**.
2. **Ver Estado de Cuenta:** Muestra su saldo disponible (**dinero**) y recorre el **Historial de Transacciones** (la **Pila**) para mostrar todas sus operaciones registradas.
3. **Gestión de Contactos:** Puede agregar o eliminar **Contactos** (otros **IDs** de cliente) para simplificar futuros **Depósitos**. Esta lista se mantiene en una **Lista Doblemente Enlazada** asociada a su cuenta.

B. Solicitud de Transacción (Formarse en la Fila)

1. Cuando selecciona "Realizar Transacción" (Retiro, Préstamo, etc.), el sistema le pedirá el tipo de servicio.
2. **Importante:** En este punto, **su dinero NO se mueve**.
3. La solicitud simplemente llama a la función **Empleado::FormarFila**, la cual toma la información de su cuenta y la añade a la **Cola (Fila)** de atención correspondiente (por ejemplo, **filaClientesRetiros**).
4. Usted queda en espera, y la transacción será procesada en orden **FIFO** (Primero en Entrar, Primero en Salir) por el **Empleado**.

3. Guía para el Empleado

El empleado es el responsable de ejecutar las operaciones bancarias y mantener la integridad de la base de datos.

A. Tareas Principales

1. **Mostrar Filas:** Permite revisar quién está esperando en cada una de las colas de servicio (Retiros, Depósitos, Préstamos y Problemas).

2. **Atender Clientes (Procesar Solicitud)**: Esta es la tarea crítica.

B. Flujo Crítico de Procesamiento (AtenderCliente)

Cuando selecciona "Atender Clientes" para una fila específica (por ejemplo, Retiros), se activa la función **Empleado::AtenderCliente**, desencadenando el siguiente proceso:

1. **Selección del Cliente**: La función **FilaClientes::EliminarClienteFila** retira al cliente que lleva más tiempo esperando de la cola (principio **FIFO**).
 2. **Ejecución de Lógica**: La función **FilaClientes::AtenderCliente** aplica la lógica financiera:
 - Para un **Retiro**, verifica el saldo y lo resta.
 - Para un **Depósito**, busca al cliente destinatario (usando **Empleado::BuscarCliente**), resta el monto de la cuenta de origen y lo suma a la cuenta de destino.
 - Para **Problemas**, se ofrece la opción de invocar **Cliente::EliminarTransaccion**, la cual **revierte el efecto de la última transacción** en el saldo del cliente.
 3. **Guardar Cambios (Persistencia)**: Una vez que el saldo y el historial del cliente se han modificado, se llama a **Empleado::ActualizarCliente**. Esta función se encarga de localizar al cliente en el **ABB** y guardar los nuevos datos de forma permanente.
-

1. System Access and Role Selection

Upon starting the application, you'll be prompted to choose your role:

- **Opción 1: Cliente**
 - Allows you to create an account or log in to manage your funds and request services.
- **Opción 2: Empleado**
 - Requires fixed credentials to access the processing panel (ID: **275460**, Password: **85696**).

2. Client Guide (Cliente)

As a client, your main task is to request services, which places your request in a processing queue.

A. Account Management (Gestión de Cuenta)

1. **Crear Cuenta**: You are assigned a unique **ID**, and your new account is stored in the central database (the **ABB**, or Binary Search Tree) via the **Empleado::InsertarCliente** function.
2. **Ver Estado de Cuenta**: Displays your current **dinero** (balance) and traverses your **Historial de Transacciones** (the **Pila** or Stack) to show all registered operations.

3. **Gestión de Contactos:** You can add or remove **Contactos** (other client IDs) using a **Lista Doblemente Enlazada** to simplify future **Depósitos** (Deposits).

B. Requesting a Transaction (Formarse en la Fila)

When you choose to perform a transaction (like "Retiro" or "Préstamo"):

1. **Important:** At this stage, **your money is NOT moved**.
2. Your request calls the **Empleado::FormarFila** function.
3. This function takes your account information and adds it to the appropriate service queue (**Fila**), such as **filaClientesRetiros**.
4. You are now in the waiting queue and must wait for an **Empleado** to process your request in **FIFO** order.

3. Employee Guide (Empleado)

The employee is responsible for the actual execution of banking operations and maintaining data integrity.

A. Main Tasks (Tareas Principales)

1. **Mostrar Filas:** Allows you to view which clients are waiting in each segmented service queue (Retiros, Depósitos, Préstamos, and Problemas).
2. **Atender Clientes (Process Request):** This is the critical task where money movement occurs.

B. Critical Processing Flow (AtenderCliente)

When you select "Atender Clientes" for a specific queue (e.g., Retiros), the **Empleado::AtenderCliente** function is activated, initiating the transaction process:

1. **Client Selection:** The **FilaClientes::EliminarClienteFila** function retrieves the client who has been waiting the longest in the queue (**FIFO** principle).
2. **Logic Execution:** The **FilaClientes::AtenderCliente** function applies the financial rules:
 - For a **Retiro**, it verifies the balance and subtracts the amount.
 - For a **Depósito**, it finds the recipient client using **Empleado::BuscarCliente**, subtracts the amount from the source, and adds it to the destination account.
 - For **Problemas**, you are given the option to invoke **Cliente::EliminarTransaccion**, which **reverses the effect of the last transaction** on the client's balance.
3. **Saving Changes (Persistencia):** After modifying the client's balance and history, **Empleado::ActualizarCliente** is automatically called. This function locates the client within the **ABB** and permanently saves the updated data.